Mini-Project Title:

Automated Terms & Conditions Summarizer and Clause Risk Analyzer

Team Details:

Sr. No.	Name	Roll Number
1	Disha Takawale	2413195
2	Prerana Patankar	2413196
3	Saloni Patil	2413210
4	Neha Maniyar	2413211

Experiment 1

Study Various Applications of NLP and NLP Tools

1. Objective

To study and conduct a comparative evaluation of foundational open-source NLP libraries (NLTK, spaCy, Hugging Face Transformers, AllenNLP) and major cloud-based NLP platforms (Google Cloud Natural Language API, AWS Comprehend, Azure Text Analytics).

2. Theory

Natural Language Processing (NLP) is an interdisciplinary field of computer science, artificial intelligence, and linguistics concerned with the interactions between computers and human language. NLP frameworks have evolved from classical rule-based and statistical systems to deep neural networks and pre-trained transformer architectures. Open-source libraries provide fine-grained algorithmic control and customizable offline pipelines, whereas enterprise cloud NLP platforms provide managed, high-throughput REST APIs for pre-trained linguistic analysis, entity extraction, and sentiment classification.

3. Study / Evaluation Framework

Category 1: Open-Source NLP Libraries

- **NLTK (Natural Language Toolkit):** A foundational Python suite for classical and statistical NLP containing over 50 corpora and lexical resources (e.g., WordNet), tokenizers, stemmers, POS taggers, and rule-based grammars.
- **spaCy:** A high-performance, production-ready NLP library written in Cython. Features fast multi-task neural network pipelines for tokenization, lemmatization, Part-of-Speech tagging, dependency parsing, and Named Entity Recognition (NER).
- **Hugging Face Transformers:** A modern deep learning ecosystem providing thousands of state-of-the-art pretrained transformer models (BERT, RoBERTa, T5, GPT, BART) for abstractive generation, classification, and sequence labeling with PyTorch/TensorFlow backends.
- **AllenNLP:** An open-source NLP research library built on PyTorch, designed specifically for deep semantic tasks such as Semantic Role Labeling (SRL), Coreference Resolution, and machine reading comprehension.

Category 2: Cloud NLP Platforms

- **Google Cloud Natural Language API:** Managed service powered by Google machine learning models offering multi-language entity recognition, sentiment analysis, content classification into 700+ categories, and syntactic analysis.
- **AWS Comprehend:** An AWS-managed NLP service that uses machine learning to discover insights, extract key phrases, identify PII (Personally Identifiable Information), detect topics, and analyze document sentiment.
- **Azure Text Analytics (Azure Cognitive Services for Language):** Microsoft Azure cloud service providing key phrase extraction, named entity recognition with domain linking (including healthcare), opinion mining, and text summarization.

4. Implementation & Comparative Analysis

The comparative study evaluated the architecture, computational efficiency, execution environment, and optimal use cases for each tool:

Tool / Platform	Type / License	Primary Strengths	Underlying Approach	Best Suited For
NLTK	Open-Source (Apache 2.0)	Pedagogical clarity, extensive corpora, rule-based grammars	Classical rule-based & statistical models	Teaching, prototyping, corpus linguistics, CFG parsing
spaCy	Open-Source (MIT)	Blazing fast execution, industrial NER, dependency parsing	Cython-optimized convolutional / transformer models	High-throughput production pipelines, entity & syntactic extraction
Hugging Face	Open-Source (Apache 2.0)	SOTA accuracy, transfer learning, huge model hub	Deep neural Transformers (Self-Attention)	Complex transfer learning, abstractive generation, fine-tuning
AllenNLP	Open-Source (Apache 2.0)	Deep semantic understanding, modular research components	PyTorch deep learning models	Academic NLP research, Coreference, Semantic Role Labeling

Google Cloud NLP	Cloud API (Proprietary)	Multi-lingual support, scalable cloud infrastructure	Large-scale proprietary Google models	Large-scale enterprise search & content categorization
AWS Comprehend	Cloud API (Proprietary)	Native AWS integration, PII redaction, topic modeling	Managed AWS machine learning pipelines	Serverless data workflows, PII compliance, customer feedback
Azure Text Analytics	Cloud API (Proprietary)	Domain-specific entity linking (Healthcare), opinion mining	Managed Microsoft cognitive language models	Enterprise business intelligence, compliance, opinion mining

5. Tools / Libraries Covered

- **Open-Source:** NLTK 3.x, spaCy 3.x, Hugging Face Transformers, AllenNLP.
- **Cloud Platforms:** Google Cloud NLP API, AWS Comprehend, Microsoft Azure Cognitive Service for Language.

6. Results / Comparative Summary

- **Algorithmic Spectrum:** Identified that NLTK excels in educational baseline modeling, spaCy dominates structured syntactic parsing at scale, and Hugging Face represents the state-of-the-art for neural semantic tasks.
- **Deployment Trade-Offs:** Cloud APIs offer zero-maintenance, serverless scalability at recurring API costs, while open-source libraries provide data privacy, zero-egress cost, and full parameter customization.

7. What You Achieved / Learned

- Acquired a systematic understanding of the modern NLP tool ecosystem across classical, neural, and managed cloud paradigms.
- Learned the architectural trade-offs between local open-source pipelines and scalable cloud NLP services.

Experiment 2

Study Various Applications of NLP and Formulate a Mini-Project Problem Statement

1. Objective

To explore foundational and modern application domains of Natural Language Processing, conduct a literature review on legal text summarization, and formulate a formal Mini-Project Problem Statement for an Automated Terms and Conditions Summarizer.

2. Theory

Natural Language Processing applications span across text generation, sequence classification, information extraction, and dialogue management. Real-world tasks range from cross-lingual Machine Translation to intelligent Chatbots and Automated Summarization. In domains where documents are excessively long and verbose—such as legal terms of service agreements—NLP provides automated mechanisms to ingest, parse, clean, categorize, and summarize critical clauses into concise, actionable representations.

3. Study of Major NLP Applications

1. **Machine Translation:** Automated translation of text between natural languages using Sequence-to-Sequence (Seq2Seq) neural networks and Attention-based Transformer models (e.g., Google Translate, DeepL).
2. **Text Categorization:** Classifying unstructured text documents into predefined thematic categories using feature representations (TF-IDF, word embeddings) and classifiers (Naive Bayes, SVM, BERT).
3. **Text Summarization:** Extractive Summarization selects and ranks the most salient sentences directly from the source text (e.g., TextRank, LexRank); Abstractive Summarization generates novel sentences that capture the core meaning using encoder-decoder transformers (e.g., BART, T5, PEGASUS).
4. **Chatbots & Conversational Agents:** Automated dialogue systems utilizing intent recognition, slot filling, and Large Language Model (LLM) prompting to resolve customer queries.
5. **Plagiarism Detection:** Determining lexical and semantic overlap between documents using N-gram matching, cosine similarity over vector embeddings, and string alignment.
6. **Grammar & Spell Checking:** Identifying syntactic and stylistic anomalies using statistical language models, Levenshtein edit distance, and sequence-to-sequence correction.

7. **Sentiment Analysis & Opinion Mining:** Determining subjective polarity (positive, negative, neutral) in user reviews and social media using lexicon-based tools (VADER) or fine-tuned transformers.
8. **Question Answering (QA):** Extracting exact answer spans or generating direct answers from reference documents using Retrieval-Augmented Generation (RAG) and dense vector search.
9. **Personal Assistants & Tutoring Systems:** Multi-modal interactive AI systems (e.g., Siri, Alexa, Duolingo) integrating Automatic Speech Recognition (ASR), dialogue state tracking, and adaptive knowledge retrieval.

4. Selected Real-World Application & Literature Review

Selected Domain: Legal Tech & Consumer Rights — Automated Terms & Conditions Summarization.

Literature Review Summary:

- Standard digital consumer agreements average 8,000 to 15,000 words in dense, complex legal terminology (legalese). Over 90% of users accept these terms without reading, forfeiting privacy rights and accepting unilateral liability waivers.
- Prior benchmark datasets like the ToS;DR (Terms of Service; Didn't Read) corpus and the LexGLUE benchmark demonstrate that legal text requires specialized handling due to long-range syntactic dependencies, domain-specific terminology, and strict requirements for semantic fidelity (where omitting words like not or without fundamentally alters legal liability).

5. Mini-Project Problem Statement & System Architecture

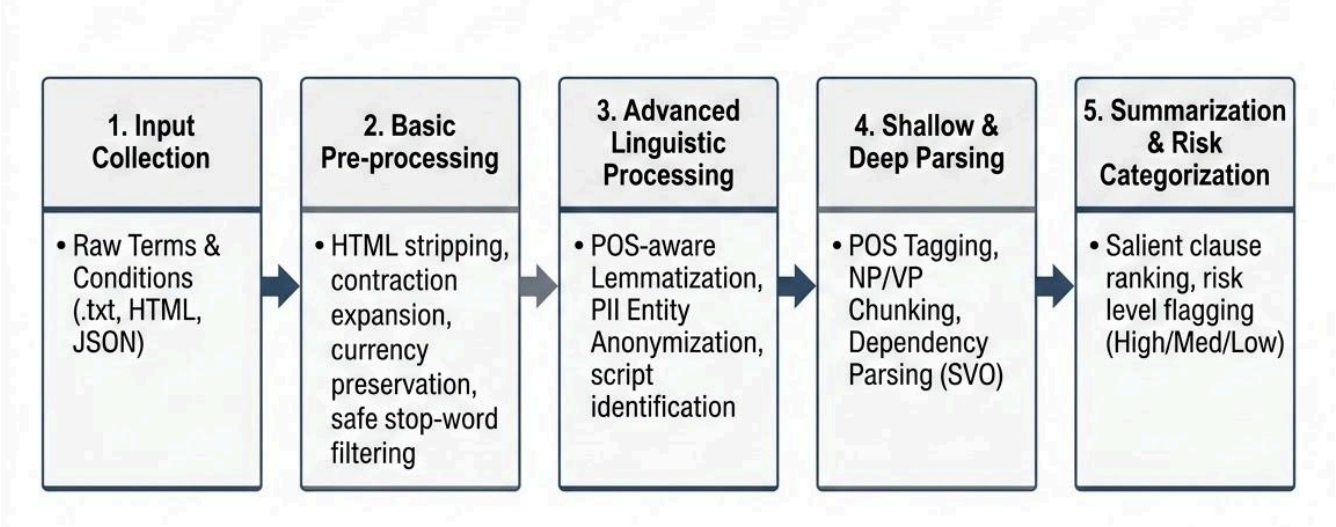
Problem Statement:

> Title: Automated Terms & Conditions Summarizer and Clause Risk Analyzer
>
> Problem Statement: Digital users routinely accept complex, multi-page Terms and Conditions without reading them due to verbose legal jargon, unclear liability waivers, and obscure data privacy clauses. This project aims to build an end-to-end NLP system that automatically ingests, cleans, segments, categorizes contractual clauses, and produces concise, human-understandable summaries while flagging critical user risks (e.g., unilateral account termination, liability limits, and arbitration mandates).

Case Study & Datasets:

Real-world Terms of Service and Free Membership Agreements from major e-commerce platforms: **Amazon Conditions of Use** (amazon.txt) and **Alibaba Free Membership Agreement** (alibaba.txt), structured into labeled clause categories (tc_clauses.csv).

System Architecture & Methodology



6. Tools / Libraries Used

Python 3.13, pandas, nltk, spacy, scikit-learn.

7. What You Achieved / Learned

- Explored the foundational breadth of NLP application domains and their practical mechanics.
- Successfully conducted a literature review on legal NLP challenges and formulated a clear, end-to-end system architecture for the Terms & Conditions Summarizer.

Experiment 3

Implement NLP Pre-processing Tasks

1. Objective

To design and implement a comprehensive NLP text pre-processing pipeline for contractual text, covering HTML stripping, special character removal, whitespace normalization, sentence segmentation, contraction expansion, number/currency handling, negation-preserving stop-word removal, phrase tokenization, and script validation.

2. Theory

Text pre-processing transforms noisy, unstructured raw text into clean, standardized units suitable for downstream NLP models. In legal documents, standard aggressive cleaning can be disastrous if critical numbers (monetary caps), percentages (liability shares), or negation tokens (not, no, never, without) are stripped. Preprocessing must therefore balance noise suppression with semantic preservation.

3. Your Case Study / Dataset

Case Study Text: Raw scraped HTML legal agreements (dirty_tc_sample.html) with embedded hyperlinks, contact emails, phone numbers, and HTML tags.

Sample Clause:

```
<p>We can't guarantee services. Under no circumstances shall liability
exceed $1,000.00 USD or 100% of fees paid.</p>
```

4. Implementation

We constructed a 10-task sequential pre-processing module:

- **HTML Removal:** BeautifulSoup(text, 'html.parser').
- **Script Validation:** Unicode normalization (unicodedata.normalize('NFKD')).
- **Contraction Expansion:** contractions.fix() (can't → cannot, won't → will not).
- **Regex Sanitization:** Stripping URLs (http\S+), emails, and helplines.
- **Number & Currency Normalization:** Preserving $1,000.00 as CURRENCY_USD_1,000.00 and 100% as 100_PERCENT.
- **Sentence Segmentation:** NLTK sent_tokenize for clause boundary detection.
- **Negation-Preserving Stop-word Filtering:** Keeping {"not", "no", "never", "without", "neither", "cannot"} while filtering non-informative words.

5. Tools / Libraries Used

- Python 3.13, BeautifulSoup4, contractions, nltk, re, unicodedata, pandas.

6. Results / Output

Preprocessing Transformation Pipeline:

Preprocessing Stage	**Input String Snippet**	**Processed Output**
Raw HTML	We can't guarantee... Contact: <a>email	We can't guarantee... Contact: email
Contraction Expansion	We can't guarantee services won't be interrupted.	We cannot guarantee services will not be interrupted.
Currency & Number Token	...shall liability exceed $1,000.00 USD or 100%...	...liability exceed CURRENCY_USD_1,000.00 or 100_PERCENT...
Sentence Segmentation	Continuous paragraph text	3 Discrete Clauses identified
Stop-word Removal (Safe)	under no circumstances shall liability exceed...	['circumstances', 'no', 'liability', 'exceed'] (Negation 'no' retained!)

7. What You Achieved / Learned

- Built a robust, reusable pre-processing pipeline tailored for legal agreements.
- Discovered that retaining domain entities (currencies, percentages, negations) is critical to preserving legal liability boundaries.

8. Google Colab Implementation

- **Google Colab Link:** Experiment_3_PreProcessing.ipynb

Experiment 4

Implement Advanced Text Pre-processing Techniques

1. Objective

To implement advanced text pre-processing and linguistic normalization techniques on Terms & Conditions agreements, including domain-specific stop-word removal, comparative Stemming vs. POS-aware Lemmatization, noise handling, multilingual language detection, and entity anonymization (PII masking).

2. Theory

Advanced pre-processing addresses linguistic variation, language diversity, and privacy regulations. Stemming applies heuristic suffix stripping which often results in non-words, while Lemmatization maps words to valid morphological dictionary roots using Part-of-Speech context. Entity anonymization (masking Personally Identifiable Information like user names, emails, and financial amounts) is legally mandatory when processing contractual agreements under privacy frameworks like GDPR and CCPA.

3. Your Case Study / Dataset

- **Data Source:** Amazon & Alibaba agreements containing legal morphology (indemnification, terminating, obligations, disclaimers) and user account records with PII (data/tc_clauses.csv).
- **Multilingual Sample:** English Amazon terms, Alibaba Chinese terms (本协议受中华人民共和国法律管辖), and Spanish terms.

4. Implementation

- **Morphological Comparison:** Benchmarked Porter Stemmer, Lancaster Stemmer, and WordNet Lemmatizer (with Noun and Verb POS tags) on legal terminology.
- **PII Masking & Entity Anonymization:** Regex + spaCy Named Entity Recognition (NER) pipeline replacing user names with <USER_NAME>, emails with <EMAIL>, phone numbers with <PHONE_NUMBER>, and monetary values with <MONETARY_AMOUNT>.
- **Multilingual Detection:** Implemented automated language identification using langdetect.

5. Tools / Libraries Used

Python 3.13, nltk (Porter, Lancaster, WordNet), spacy (en_core_web_sm), langdetect, pandas.

6. Results / Output

1. Stemming vs Lemmatization on Legal Morphology:

Original Legal Term	Porter Stemmer	Lancaster Stemmer	WordNet Lemma (Noun)	WordNet Lemma (Verb)
indemnification	indemnif (invalid)	indemn	indemnification (valid noun)	indemnification
terminating	termin	termin	terminating	terminate (root verb)
obligations	oblig	oblig	obligation (singular form)	obligations
breaching	breach	breach	breaching	breach (root verb)

2. Entity Anonymization (PII Redaction) Output:

- **Raw Input:** "User John Doe agreed to pay $2,500.00 to Alibaba Services LLC. Contact: jdoe@sample.com or call +1 800 555 0199."
- **Anonymized Output:** "User <USER_NAME> agreed to pay <MONETARY_AMOUNT> to Alibaba Services LLC. Contact: <EMAIL> or call <PHONE_NUMBER>."

3. Multilingual Detection:

- English Terms: en (100% confidence)
- Alibaba Chinese Clause (本协议受中华人民共和国...): zh-cn (100% confidence)

7. What You Achieved / Learned

- Proved that POS-aware Lemmatization is significantly superior to Stemming for legal text as it preserves valid dictionary lemmas required for semantic matching.
- Implemented an automated privacy-preserving PII redaction pipeline ensuring GDPR-compliant contract ingestion.

8. Google Colab Implementation

- **Google Colab Link:** Experiment_4_Advanced_PreProcessing.ipynb

Experiment 5

Implement Shallow and Deep Parsing

1. Objective

To implement Part-of-Speech (POS) Tagging, Shallow Parsing (NP/VP Chunking for rights and obligations), Deep Dependency Parsing (Subject-Verb-Object relation extraction), and Context-Free Grammar (CFG) Parse Tree generation on contractual clauses.

2. Theory

Syntactic parsing analyzes the hierarchical and grammatical structure of sentences. Shallow parsing (chunking) groups adjacent words into non-overlapping grammatical phrases such as Noun Phrases (NP) and Verb Phrases (VP). Deep parsing (dependency parsing) constructs directed grammatical graphs showing relationships between head words and dependents (e.g., subjects, root verbs, direct objects). CFGs provide formal rule-based grammars to generate hierarchical parse trees representing contract sentence validity.

3. Your Case Study / Dataset

- **Clause 1 (Obligation):** "The customer must maintain strict confidentiality of their account password."
- **Clause 2 (Disclaimer):** "Amazon Services LLC disclaims all warranties under Section 12."

4. Implementation

- **POS Tagging:** Applied NLTK Perceptron Tagger to label determiners (DT), modals (MD), verbs (VB), and nouns (NN).
- **Shallow Parsing / Chunking:** Formulated a regular expression grammar:
    - NP: {<DT|PRP$>?<JJ.*>*<NN.*>+} (Actors, entities, and contractual assets)
    - VP: {<MD>?<VB.*>+} (Modal obligations and actionable verbs)
- **Deep Dependency Parsing:** Parsed sentences with spaCy to extract Subject-Verb-Object (SVO) triples (Actor → Action → Target).
- **CFG Parse Tree Generation:** Defined a custom legal Context-Free Grammar (nltk.CFG) and generated formal syntax parse trees.

5. Tools / Libraries Used

Python 3.13, nltk (pos_tag, RegexpParser, CFG, ChartParser), spacy (en_core_web_sm), pandas.

6. Results / Output

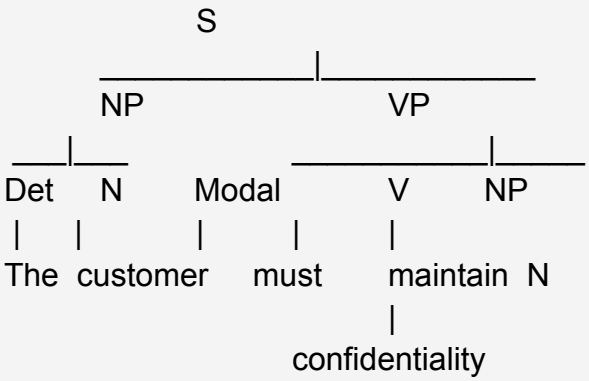
1. Shallow Chunking Extraction:

```
[NP] -> The customer            (Contractual Actor)
[VP] -> must maintain           (Mandatory Legal Obligation)
[NP] -> strict confidentiality   (Object of Obligation)
[NP] -> their account password   (Target Asset)
```

2. Deep Dependency SVO (Semantic Relation Extraction):

- **Actor / Subject (nsubj):** ['Amazon Services LLC']
- **Action / Root Verb (ROOT):** ['disclaims']
- **Target / Direct Object (dobj):** ['warranties']

3. Context-Free Grammar (CFG) Hierarchical Parse Tree:



7. What You Achieved / Learned

- Mastered extracting structural legal semantics: using shallow chunking to isolate obligations ("must maintain") and deep dependency parsing to extract Who disclaims What (Amazon → disclaims → warranties).
- Generated formal CFG parse trees visualizing the exact syntactic hierarchy of contract sentences.

8. Google Colab Implementation

Google Colab Link: Experiment_5_Parsing_and_Trees.ipynb